Indy Architecture & Technical Specification

1. Executive Summary

Indy is a Chrome extension (Manifest V3) and companion Native Messaging / HTTP bridge that makes accepting root and intermediary Transport Layer Security (TLS) certificates visible and optional.

Modern web browsers blindly trust hundreds of pre-installed root Certificate Authorities (CAs) shipped by operating systems and root store programs. When an HTTPS connection is negotiated, the browser silently validates the certificate chain against these trust anchors without alerting the user or offering policy choices.

Indy inverts this paradigm:

1. It boots a local policy database directly from the host operating system's system root certificate store (e.g. `/etc/pki/ca-trust/extracted/pem/tls-ca-bundle.pem` on Fedora/RHEL or `/etc/ssl/certs/ca-certificates.crt` on Debian/Ubuntu).
2. It tracks both Root CAs and Intermediary CAs in a persistent hierarchy.
3. When an incoming certificate chain anchors to a Root CA (such as `indy` or commercial roots) or an Intermediary CA that is not explicitly marked as `accept`, Indy intercepts the navigation.
4. Indy presents an interstitial prompt displaying the full certificate chain hierarchy, allowing the user to select one of three policy actions:
    - Accept: Trust the certificate authority automatically on all future requests.
    - Ask: Prompt the user every time a connection chains to this authority (with one-time bypass for the current visit).
    - Deny: Refuse connections chaining to this authority and block domain subresources via Declarative Net Request (DNR) session rules.

---

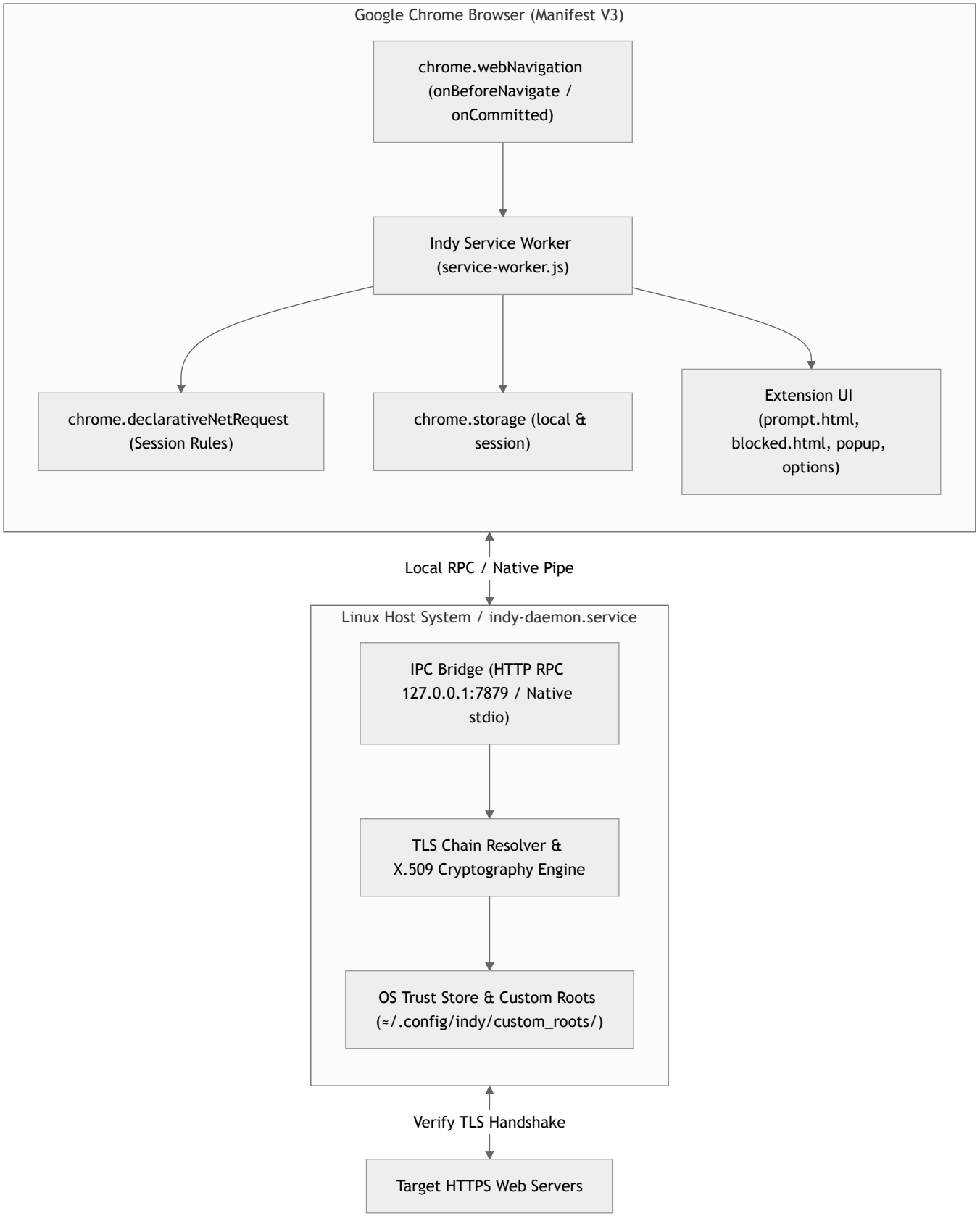
2. Core Constraints & Technical Architecture

2.1 Manifest V3 Sandboxing & Host Bridge

Google Chrome Manifest V3 sandbox rules impose two key security boundaries:

- Extensions cannot read the host filesystem or execute local trust store utilities (`trust list`, `openssl`).
- Chrome's `chrome.webRequest` API does not expose X.509 certificate chains to extension JavaScript (unlike Firefox's `browser.webRequest.getSecurityInfo()`).

To solve these constraints cleanly across standard Chrome, Chromium, and Flatpak Chrome (`com.google.ChromeDev`), Indy implements a dual-transport host architecture:



2.2 Dual Transport Protocol (HTTP RPC & Native Messaging)

1. Local HTTP RPC (`http://127.0.0.1:7879/rpc`):

    - Preferred for Flatpak Chrome where containerized processes share the host network stack (`shared=network`) but cannot spawn arbitrary host binaries.
    - Maintained by a background systemd user service (`indy-daemon.service`).
2. Native Messaging Host (`com.etu.indy`):
    - Uses the standard 4-byte length-prefixed stdio protocol.
    - Automatically loaded when running inside native Chrome or Chromium packaging.

---

3. Data Models & Certificate Database Schema

Persistent records reside in `chrome.storage.local`. Ephemeral per-tab states and short-lived TLS caches reside in `chrome.storage.session`.

3.1 Certificate Record (`certDatabase[id]`)


```
{
  "id": "sha256:749ed88a433149e1556befd993183055feee2c613a6ce38537088fd857ef2638",
  "fingerprint": "749ED88A433149E1556BEFD993183055FEEE2C613A6CE38537088FD857EF2638",
  "commonName": "indy",
  "organization": "Indy Root Certificate Authority",
  "country": "FI",
  "subject": "CN=indy,O=Indy Root Certificate Authority,C=FI",
  "issuerCommonName": "indy",
  "issuerOrganization": "Indy Root Certificate Authority",
  "issuer": "CN=indy,O=Indy Root Certificate Authority,C=FI",
  "role": "root",
  "isCA": true,
  "isSelfSigned": true,
  "subjectKeyId": "314404E76B98A71BEDC2F830B8CE21A07440E2F9",
  "authorityKeyId": "314404E76B98A71BEDC2F830B8CE21A07440E2F9",
  "serialNumber": "010000000000000001",
  "signatureAlgorithm": "sha256WithRSAEncryption",
  "notBefore": "2025-01-01T00:00:00+00:00",
  "notAfter": "2035-01-01T00:00:00+00:00",
  "source": "system",
  "policy": "ask",
  "firstSeenAt": "2026-10-22T00:00:00.000Z",
  "updatedAt": "2026-10-22T00:00:00.000Z"
}
```


3.2 Certificate Roles and Relations

- `role: "root"`: Trust anchor certificate. Typically self-signed (`subject == issuer`) or directly derived from the operating system's trust store.
- `role: "intermediate"`: Intermediate CA certificate issued by a Root CA or higher-level Intermediate CA. Contains a `parentId` field referencing its issuer's SHA-256 fingerprint ID.

- `role: "leaf"`: End-entity TLS certificate issued to the web host. Not stored permanently in `certDatabase`, but inspected dynamically during navigations.

3.3 Private & Custom Root Certificate Ingestion

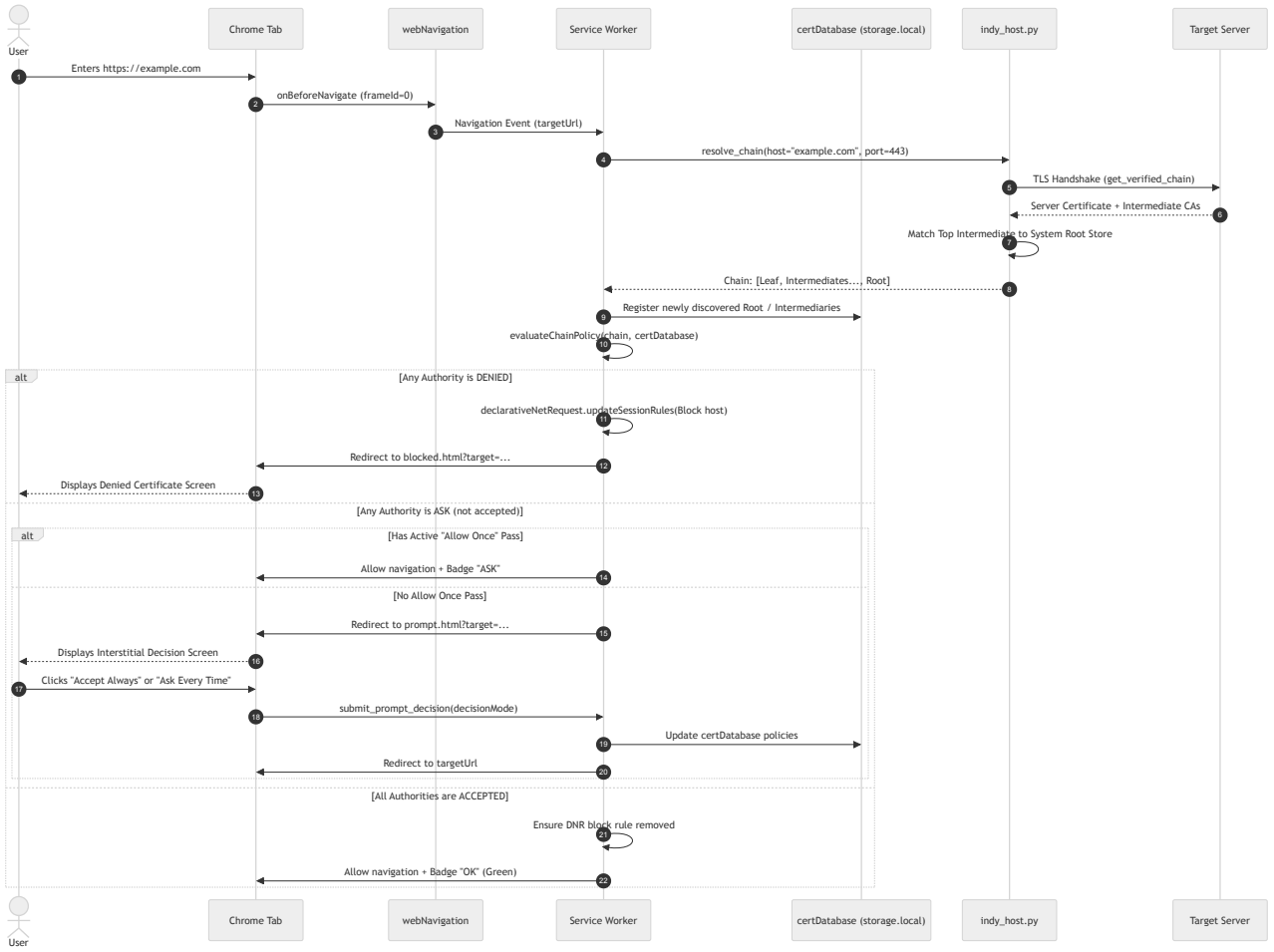
In addition to system trust stores, Indy supports provisioning private, enterprise internal, and development root CAs:

1. Upload & Parsing: Users upload or paste PEM-encoded X.509 certificates (`.pem`, `.crt`, `.cer`) via the Options UI modal. The service worker delegates parsing to `indy_host.py` via `parse_certificate`, which uses the Python `cryptography.x509` engine to extract Subject, Issuer, Serial Number, Basic Constraints (`isCA`), and SHA-256 fingerprint without persisting anything prematurely.
2. Host Persistence (`≈/.config/indy/custom_roots/`): Upon user confirmation (`upload_root_certificate`), the host daemon writes the PEM data to `$HOME/.config/indy/custom_roots/<FINGERPRINT>.pem` and registers it directly into `indy_host.py` 's live authority index (`_roots_by_id`, `_roots_by_ski`, `_roots_by_subject`).
3. Browser State Synchronization: The background service worker registers the new authority in `chrome.storage.local.certDatabase` with `source: "custom"` and the user's initial policy choice (`ask`, `accept`, `deny`). Custom roots are dynamically displayed with a distinct `Private CA` badge in the database manager and can be deleted on-demand with one click (`delete_custom_root`), cleanly removing the host PEM file and browser policy record.

---

4. Policy Evaluation & Interception Lifecycle

When the user navigates to an HTTPS website, Indy evaluates the chain in real time before page content loads:



4.1 Evaluation Rules & Precedence

1. Rule 1 - Denial Precedence: If any certificate in the authority chain (Root CA or any Intermediate CA) is set to `deny`, the entire connection is rejected immediately.
2. Rule 2 - Intermediary Enforcement: When `enforceIntermediates` is enabled (default), both Root and Intermediate CAs must have `policy === "accept"` for automated pass-through. If an accepted Root CA has an intermediate with `policy === "ask"`, the user is prompted specifically for that intermediate.
3. Rule 3 - Acceptance Criteria: Only when all authority certificates in the chain are marked as `accept` does Indy permit the connection without user friction.

---

5. Security & Isolation Considerations

- Content Security Policy (CSP): Extension pages enforce strict MV3 CSP. No inline scripts, no `eval()`, and no remote code execution. All dynamic DOM nodes are created safely via `document.createElement()` and `textContent`.
- Network Isolation via DNR: When a certificate is denied, Indy doesn't simply redirect the main HTML document; it injects a Declarative Net Request session rule blocking subresources (scripts, XMLHttpRequests, stylesheets, fonts, websockets) to prevent covert data leakage.

- Deterministic Identity: The extension manifest specifies a deterministic 2048-bit RSA public key, locking the extension ID across all installations and test environments to `hmfkankhnneijoaagpgpdcccmoadjaep`.

---

6. Division of Responsibility: Indy Policy Gating vs. Native Chrome Certificate Engine

A critical architectural distinction in Indy is that it does **not** replace or patch Google Chrome's underlying C++ cryptographic certificate verifier (`//net/cert/CertVerifyProc` and BoringSSL). Instead, Indy operates as an **outer supervisory policy and authorization gate** layered on top of Chrome's native network stack.

6.1 Native Chrome Cryptographic Engine Responsibilities

The native Chrome network engine remains strictly authoritative for the cryptographic validity and integrity of every TLS connection:

1. Cryptographic Signature Validation: BoringSSL verifies that each certificate in the TLS chain is cryptographically signed by the public key of its issuing Certificate Authority using valid signature algorithms (e.g. SHA-256 with RSA, ECDSA P-256/P-384).
2. Hostname & SAN Matching: Chrome validates that the target domain in the URL matches the Subject Alternative Name (`dNSName` or `iPAddress`) entries in the leaf certificate. If there is a mismatch, Chrome natively aborts with `NET::ERR_CERT_COMMON_NAME_INVALID`.
3. Validity Period & Expiration: Chrome enforces certificate lifetime boundaries (`notBefore` and `notAfter`) against the local system clock. Expired or not-yet-valid certificates fail natively with `NET::ERR_CERT_DATE_INVALID`.
4. Certificate Revocation & CRLSets: Chrome checks revocation state using out-of-band compiled CRLSets and OCSP stapling. Revoked certificates fail natively with `NET::ERR_CERT_REVOKED`.
5. Basic Constraints & Key Usages: Chrome enforces that intermediate and root CAs have `basicConstraints: CA:TRUE`, valid path lengths, and appropriate `keyUsage` flags (`keyCertSign`, `cRLSign`, `serverAuth`).
6. Native Trust Store Verification: Chrome verifies that the chain anchors to a recognized trust anchor in the Chrome Root Store (CRS) or local OS database (e.g. NSS database `$HOME/.pki/nssdb` or system `/etc/pki/ca-trust`).

> **NOTE**
>
> Chrome's native security baseline remains active at all times. If a certificate chain is forged, expired, or revoked, Chrome's internal net stack blocks the connection immediately. Indy cannot and does not bypass or override native cryptographic verification errors.

6.2 Indy Extension Supervisory Responsibilities

Chrome's native engine is binary: if a certificate chain is valid and anchors to *any* pre-installed root, Chrome silently accepts the connection without user notification or policy options. Indy intercepts this silent acceptance:

1. Elimination of Silent Trust: Chrome silently trusts hundreds of commercial and governmental root CAs. Indy checks every chain against user-configured policies (`accept` , `ask` , `deny`), eliminating silent pass-through.
2. Interactive Interstitial Prompts (`prompt.html`): When a certificate chain anchors to an authority with policy `ask` , Indy pauses the browser navigation before page content loads, rendering the complete chain hierarchy (Root → Intermediaries → Leaf) for explicit user review.
3. Selective Intermediate CA Authorization: Chrome treats any intermediate CA issued by a trusted root as valid. Indy allows users to isolate intermediate CAs (e.g., trust commercial Root CA `DigiCert Global Root G2` , but selectively deny or prompt for specific enterprise intermediate CAs).
4. Subresource Request Blocking via DNR: When an authority is set to `deny` , Indy prevents covert background subresource leaks (scripts, images, fetch calls, websockets) by dynamically registering Declarative Net Request (DNR) session rules for the target origin.
5. Host Root Store Sync & Custom Root Provisioning: Indy synchronizes system root certificates into an accessible local database and enables importing private, homelab, and enterprise internal root certificates with live X.509 metadata preview.

6.3 Functional Comparison Matrix

SECURITY / TRUST FUNCTION	HANDLED BY NATIVE CHROME ENGINE	HANDLED BY INDY	OPERATIONAL CONSEQUENCE
RSA / ECDSA Signature Math	Yes	No	Chrome blocks mathematical forgery (`ERR_CERT_INVALID`).
Domain / SAN Validation	Yes	No	Chrome blocks domain impersonation (`ERR_CERT_COMMON_NAME_INVALID`).
Certificate Expiration Dates	Yes	No	Chrome blocks expired certs (`ERR_CERT_DATE_INVALID`).
Revocation Checks (CRLSets)	Yes	No	Chrome blocks revoked certificates (`ERR_CERT_REVOKED`).
Silent Trust Elimination	No (Silently accepts)	**Yes**	Indy halts automated connection for non-accepted CAs.
Root & Intermediate Gating	No (All or nothing)	**Yes**	User controls each CA level individually (`accept` , `ask` , `deny`).
User Interstitial Decision	No (Only on error)	**Yes**	User can choose Accept Always, Ask Every Time, or Deny.
Subresource Leak Isolation	No	**Yes**	DNR session rules block background network traffic on Deny.
Private Root Ingestion & Preview	No UI / Hidden in chrome://settings	**Yes**	Ingest custom roots with live metadata preview & policy assignment.

6.4 Practical Security Considerations

- **Dual-Approval Requirement for Private Roots**: When a user uploads a new private root CA in Indy and sets its policy to `accept` , Chrome's native engine will still require that root certificate to be registered in the OS or NSS trust store (`certutil -d sql:$HOME/.pki/nssdb -A ...`). If not installed in the OS store, Chrome will correctly display `NET::ERR_CERT_AUTHORITY_INVALID` .
- **Pre-Emptive Navigation Interception**: Indy intercepts top-level navigations at `webNavigation.onBeforeNavigate` and `onCommitted` . If an authority is denied, Indy redirects the tab to `blocked.html` before any server HTML or scripts execute, preventing browser fingerprinting and zero-day payload delivery.